

DCM Test 3

March 9th

What can be virtualized?

CPU

Network

OS

All. of the above

None of the above

Computer Organization

- Program Counter
 - Holds the memory address of the next instruction to be fetched and executed
 - Automatically increments after instruction execution
 - Updated to point to the following instruction after an instruction is executed
- Memory
 - Hold both data and instructions to be executed
- Instruction Decoder
 - Translates the binary opcode into a set of control signals once the CPU fetches an instruction from memory
- ALU (Arithmetic Logic Unit)
 - To perform arithmetic and logical operations
 - Addition, subtraction, multiplication, division, and logical comparisons such as AND, OR, and NOT

Arithmetic Logic Unit

- The Arithmetic logic unit is a multi-function register that performs all the arithmetic and logical (Boolean) operations necessary to implement the instruction set
- The functions and corresponding control signals are required are as follows
 - ALE: ALU Enable
 - ALX ALU "X" function select
 - ALY: ALU "Y" Function select
 - AOE: A register tri state output enable

Instruction Set Architecture

- ISA defines the interface between software and hardware by specifying the supported instructions, data types, registers, addressing modes, and memory architecture.
 - ISA acts as a contract between the software (like compilers and OS) and the hardware (CPU)
 - Abstract the hardware so that they can be controlled by software
- Assembly
 - Low Level Programming Language that provides a more human-readable way to write machine code
 - ADD, SUB, LDA, AND, OUT, etc

What is an Operating System?

- ISA sets the rules for what the processor can do. And the OS builds on these rules to manage the hardware, ensure security, and provide a user-friendly interface to the computer system
- Program that acts as an intermediary between a user of a computer and the computer hardware
 - Software that converts hardware into a useful form for applications
 - Not easy to define precisely, no universal definition.

System Operation

- OS controls
 - One or more CPUs, GPUs, and computer accelerators
 - I/O: Disks, Network interfaces, Mouse NO, Keyboard NO, printer NO, etc
 - Displays NO
 - NO stands for X emoji

Purpose of OS

- Serves as the intermediary between computer hardware and user applications
 - Resource management: Efficiently managing hardware resources such as the CPU, memory, and storage to ensure that each application gets the resources it needs.
 - User Interface: Providing an interface (graphical or command line) that allows users to interact with the computer system
 - Task Scheduling and Multitasking: Enabling multiple applications to run concurrently by managing the execution of processes.
 - Security and stability: Protecting the system from unauthorized access and ensuring that programs run without interfering with one another
 - File management: Organizing and managing files and directories to provide easy access and storage for data.

- Provides set of utilities to simplify application development

Role of OS

- Abstract resources
 - standard libraries
 - control anything valuable (CPU memory disks etc)
- Advantage of standard library
 - allow applications to reuse common functionalities
 - make different hardware devices look the same
- Challenges
 - What are the correct abstraction
 - how much hardware should be exposed

Abstraction

- processes: illusion of being the only job in the system
- Thread: Various workers in the same processor
- Memory: virtual memory isolation processes
- File system: storage
- Message passing: Communication between different processes

Mechanisms and Policies

- Operating system = mechanisms + policies
 - mechanisms: data structures and operations that implement abstraction
 - Policies: the procedures that guide the selection of a certain course of action from alternatives
- Separate mechanisms from policies as much as possible

Architecture support for OS

- Mode of execution (user mode vs kernel mode, trap protected)
- Memory protection (paging, segmentation, virtual memory)
- Interrupts and exceptions (OS makes interruptions, software makes exceptions)
- I/O control operations (asynchronous scheduling)
- Timer operations (error scheduling)
 - timer interrupt: process running -> timer interrupt -> OS scheduler -> switch process
 - process A -> 5ms; process B -> 5ms; process C -> 5ms.

Why Privileged instruction?

- people (bad programmers) make mistakes
 - disable interrupts: overwrite memory; access disk
 - leads to: system crash, data corruption; security issues
 - OS needs self protection
- User mode -> computation; kernel mode -> control hardware
 - use system calls
 - trap to kernel mode

System Traps

- privileged instructions (in kernel mode)
 - I/O instructions
 - context switching, memory management
 - interrupt
- Non-privileged instructions (in user mode)
 - reading system time, status of processor
 - arithmetic operations
 - user mode libraries: glibc

System Traps vs Interrupt

- Traps are raised by the user program to invoke a functionality of the operating system
 - printf
 - invoke the write function to print the output to the standard output which is the monitor
 - user mode changes to kernel mode
 - After completing the task, the control is transferred back to the user mode from the kernel mode
- an interrupt is raised by a hardware device:
 - devices such as keyboards are connected to the processor via the interrupt pin. When a key is pressed, it will generate an interrupt.
 - switch from currently running process into an interrupt handler routine
 - May need to trap to kernel mode to display the pressed key on the screen
 - After completing the interrupt handler routine, the processor switches back to the original program that has been running

OS in the cloud

- Goals:
 - execute user programs and make solving user problems easier
 - make computer system convenient to use

- use the computer hardware in an efficient manner
- User want convenience, ease of use, and good performance
 - user don't care about resource utilization, or resource management
- Shared computers must keep all users happy
 - efficient and fair resource allocation and program control
 - efficient resource sharing
 - optimized for performance

User Management in OS

- multiple users in the same system
 - same set of resources
 - shared resource (users can access to all the resources)
 - User authentication and authorization rather than isolation
 - system admin has highest authority on everyone on the system
- not ideal for cloud computing

Virtualization:

- Create virtual/logic version of computers
- Main purpose
 - Abstraction: to simplify the use of the underlying resources (removing details of the structure)
 - Replications and resources sharing: to create multiple instances of the resources
 - Isolation: to separate the uses which clients make of the underlying resources

March 11th notes: From alexa :D

Virtualization

OS Virtualization:

- Creating a virtual version of machines, memory, networks, etc
- Virtualization refers to Operating System Virtualization

Virtual Machines:

- A virtualization system that partitions a single physical machine into multiple virtual machines
- Became very popular in the 2000s with server virtualization
- A virtual machine (VM) is a software-based emulation of a computer system that provides the functionality of a physical computer.

- Hardware Emulation: A VM simulates the hardware environment so that an operation system and application can run as if they were on a physical machine
- Isolation: VMs operate in isolated environments. This isolation means that processes running in one VM does not affect others, which enhances security and stability
- Resource Allocation: Virtual machines share the physical resources of the host machine. A hypervisor manages the distribution and allocation of resources among multiple VMs
- Flexibility and Portability: VMs allow for easy creation, replication, and migration of environments. This makes them valuable for testing, development, and cloud computing where workloads can be dynamically managed

Virtual Machine Monitor (Hypervisor):

- Also called a hypervisor or virtual machine manager (not in research)
 - Software that provides an interface identical to the underlying base software
 - Runs many VMs on top of a single piece of hardware
 - Resources of physical computer are shared among virtual machines
 - Each VM has the illusion of owning a complete machine and running directly on hardware
 - Host: The machines and/or software on which the VMM (hypervisor) is implemented
 - Guest: The VMs which run/execute under the control of the hypervisor
 - Bare-Metal
 - Type 1 Hypervisor
 - Physical host -> Hypervisor -> VMs -> Guest OS -> Applications
 - Complete control over the hardware
 - Hosted
 - Type 2 Hypervisor
 - Physical host -> Host operating system -> Hypervisor -> VMs -> Guest OS -> Applications
 - Avoid code duplication: no need for a process scheduler or memory management
 - Runs native processes alongside VMs
 - Easy management
- What does the Hypervisor Virtualize:
- Resource Management
 - CPU scheduling
 - Memory allocation
 - Storage virtualization

- Network virtualization
- Isolation:
- One VM crashing does not impact the rest
- Security boundaries
- Privilege Levels with Virtualization
- Traditional OS: Ring 0 -> kernel, Ring 3 -> User
- With virtualization: Ring -1 -> Hypervisor, Ring 0 -> Guest kernel, Ring 3 -> Applications

Virtualization Approaches:

- Interpretation
- Inspect each instruction in software and realize its intended effects using software
- Too slow
- Direct Execution/Interposition
- Directly executing non-privileged VM code at the CPU to attain high performance
- For any privileged VM code, VMM intercepts these attempts to perform privileged operations and retains the control of the machine

How does a VM work?

- Guest OS runs in user mode
- When an interrupt/exception/trap occurs, the hypervisor takes over
- The hypervisor simulates the hardware state changes, restarts the VM at the guest OS handler address
- Hypervisor does not trust the guest OS
- Only the hypervisor runs in full privilege in the host machine
- When the guest os tries to execute a privileged instruction, the hypervisor takes control and simulates the effect of a privileged instruction
- Only privileged instructions are trapped and emulated by the VM, most instructions are executed natively on the CPU

VM Transparency

- Full transparency:
- Stock OS runs with VM without changes
- Ex: VMWare
- Less than full transparency:
- Modified OS runs within the VM
- Better performance and simplicity
- Ex: Xen

Security

- Unprivileged code runs directly on hardware
- How do we know it's legitimate?
- Privileged code is run by the hypervisor
- How do we know it's harmless?
- How do you monitor the guest's behavior with privacy in mind?
- If the guest runs an unmodified OS, there is no way to regulate or monitor them from the inside
- Guests don't want others to know what they are doing

Virtual Machine Introspection

- Software to inspect a virtual machine from the outside for the purpose of analysing the software running inside
- External Monitoring: VMI provides insight into a VM's memory, processes, and state without needing to run any monitoring software inside the guest OS.
- Minimizes the risk of interference or compromise
- Security and Forensics: Often used for security purposes, such as detecting malware, monitoring system integrity, and performing forensic analysis
- Since it operates from the outside, it can observe behavior that might be hidden from internal monitoring tools
- Debugging and Management: VMI can also be used to debug or manage a system by inspecting its state at various points, helping developers and administrators understand system behavior

Threat Model:

- Guest OS might be compromised
 - Zero trust
 - Code running inside the guest os may be malicious
 - Inferring guest os state based on the prior knowledge of its data structures
- #### Intrusion Detection System (IDS)

- Host-based
- High visibility, low attack resistance
- Network-based
- Poor visibility, high attack resistance
- VMI IDS
- Best of both worlds
- Observes
- Memory
- CPU registers
- System calls
- Kernel data structures

- Process lists
- Monitors
- Memory access
- System calls (the event itself)
- Interrupts
- Page faults
- OS interface library
- Interprets the hardware state into OS-level events
- Policy engine
- Policy framework: defines security policies
- Policy modules: implements the security policies
- Commands
- Inspection
- Examines the VM state
- Monitoring
- Request notifications when certain events occur
- Administrative
- Controls the VM execution
- Polling Modules
- Periodically checks the system for signs of malicious activity
- Lie Detector
- Inspects the hardware and kernel state, and by querying the host system through user-level programs it detects if there is a conflict between the two views
- User Program Integrity Detector
- Periodically taking a secure hash of the immutable sections of a running program and comparing it to a known good hash
- Signature Detector
- Scanning the file system for the presence of a known malicious program
- Raw Socket Detector
- Monitoring the kernel about the type of all sockets held by user processes
- Event Driven Policy Modules
- Run when the hypervisor detects changes to the hardware state
- Memory Access Enforcer

- Marking sensitive portions of the kernel as read-only through the hypervisor, then detect modification attempts
- NIC Access Enforcer
- Prevents the ethernet device entering promiscuous mode, or being configured with a MAC address which has not been pre-specified
- Weaknesses:
 - Guest OS is able to detect the hypervisor and VMI
 - Guest OS may be able to attack the hypervisor
 - Fool the IDS
 - Disguise the bad behavior/intentionally inject detectable behavior
 - Control the behavior of the IDS and hypervisor
 - Compromise the OS library and policy engine

March 16th notes - Thanks Alexa!!

3/16/26

Pros of VMs:

- Strong Isolation: Complete OS per VM improves security
- OS Flexibility: Run different OS
- Mature Ecosystem: Well established tools

Cons of VMs:

- High Overhead: More CPU, memory, and storage
- Slow Boot: longer Startup times
- Maintenance Load: Patching multiple OSs

Hypervisor -> Docker

Applications -> Containers

Containers:

- A container is a lightweight, standalone environment that packages an application along with everything it needs to run, such as libraries and dependencies
- Multiple copies can run on a single machine
- OS resources can be partitioned
- Can be created, stopped, saved, and moved easily

Pros of Containers:

- Lightweight: Shares host OS for fast startup
- Efficient: Uses fewer resources
- Portable: Consistent across environments

Cons of Containers:

- Weaker Isolation: Shared OS kernel reduces separation
- OS Limitation: Tied to host OS type
- Newer Tech: Best practices are still evolving

Docker is a good microservice:

- Its small and focused on doing one thing
- Consistency across environments
- Isolation and fault tolerance
- Portability
- Efficient scaling
- Technology flexibility

- Streamlined CI/CD and DevOps

Cloud Container Services:

- Google is the world's largest linux container user
- Other container providers:
- AWS ECS (Elastic Container Services)
- Microsoft Azure Container Services
- IBM Bluemix Container Services

Docker

- Docker is an open-source platform for containerization. It allows developers to package applications and their dependencies into portable, lightweight containers
- Provides isolation and resource management in Host OS
- Docker Engine: Runtime
- Two editions:
- Community edition -> free and open source
- Enterprise edition -> paid and includes extra features like security, support, management
- Written in GO
- Architecture:
- Docker Client -> CLI
- Docker Daemon -> API & other stuff
- Containerd -> Container life cycle management
- Shim -> enables daemonless containers
- Runc -> container runtime (interface to kernel primitives)

- Running Containers

Docker Compose File:

- Docker image holds the task
- Properties Include:
- Ports to expose
- Network to use
- Volume to use
- Etc

Docker Swarm:

- Built-In Integration
- Simple Setup
- Manager and Worker Model
- Basic Features
- Smaller Projects
- Swarm:
- A container orchestration tool
- Consists of servers and nodes
- Manages desired state of services
- Networks span the whole swarm
- Node:
- A single server joined into the swarm
- Manager, worker, or both
- Managers check and manipulate the swarm state, worker nodes host containers
- Services:
- Definition of content managed by the swarm
- Defined similarly to a single container
- Container contains a service, a node contains a container, and it's all in the swarm

Docker Swarm Deployment:

- Specified in the deploy section
- Number of replicas/instances
- Restart policy
- Resource limits
- Each node in swarm accepts connections on published ports for any service in the swarm
- Docker swarm has its own DNS

- Each container on the network is assigned the service name as the hostname
- Kubernetes:
 - Too much for one lecture
 - Container orchestration tool
 - Advanced container orchestration
 - Complex setup
 - Master and worker architecture
 - Rich feature set
 - Large projects
 - Large community
- Container Registry Typosquatting:
 - Container Image FQID (Fully Qualified Image Identification)
 - Uniquely identify an image
 - Distinguishes images among many registries, usernames, and image names
 - registry_name/username/image_name
 - Downloading an image:
 - Docker CLI -> docker pull
 - Dockerfile -> FROM statement
 - Both has the user manually type the FQID of the desired image
 - 416,000 container image registries and 246,000 usernames in docker hub
 - 75,000 typosquatting username pairs on docker hub
 - EX:
 - Normally:
 - Docker pull reg.com/alice/linux
 - Whoopsie:
 - Docker pull reg.com/a1ice/linux
 - Docker pull regg.com/alice/linu

March 18

Disk Arrays: An easy to manage pool of disks with redundancy in datacenters, all disks are external to the server
 Data accessible by other servers in case of a server failure
 JBODs (just a bnch of disks): difficult to manage

Data access methods:

Block Access: A fixed number of bytes (block size) 1 sector, 4 sectors, 16 sectors

File access: a set of bytes with name, creation date, and other meta data.

- may or may not be contiguous
- a file system, such as, FAT32 (File allocation table) or NTFS (new technology file system) defines how the meta-data is stored and files are organized.
- file systems vary with the operating systems

Record access:: used for highly structured data in databases. each record has a particular format and set of fields. Accessed using structured query language (SQL), Open database connectivity (ODBC), Java database connectivity (JDBC)

Storage systems provide block access. a logical volume manager in the OS provides other "virtual" views, e.g. file or record.

Small Computer System Interface (SCSI)

- used to connect disk drives and tapes to computer
 - 8-16 devices on a single bus. Any number of hosts on the bus. Require host bus adapter (HBA)
 - standard commands, protocols, and optical and electrical interfaces
- Peer-to-peerL host-to-device, device-to-device, host-to-host
 - but most devices implement only targets. Can't be initiators.
 - each device on the SCSI bus has a "ID"
 - A direct access (disk) storage is addressed by a logical block address (LBA), each LB is typically 512 bytes.

Advanced Technology Attachment (ATA)

- parallel advanced technology attachment (PATA):
 - designed in 1986 for PCs. Controller integrated in the disk: integrated device electronics (IDE)
 - 133 Mbps using parallel ribbon cables
- ATA packet interface (ATAPI): Extended PATA to CDROMS, DVD-ROMs, and Tape drives
- Serial advanced Technology attachment (SATA)
 - Designed in 2003 for internal hard disks, 6 Gbps.
- PATA enhancements: ATA-2 (Ultra ATA), ATA-3 (EIDE)
- SATA Enhancements: external SATA (eSATA), mini SATA (mSATA)

How to combine multiple disks into one logical disk?

Virtual Storage:

- Partitions and file systems are "virtual" views of the storage
 - a disk array can be partitioned into virtual (logical) devices with logical unit numbers (LUNs), File systems assigned to different tenants.
- Thin provisioning: allocate blocks only when used
- Use multiple physical disks to look like a single disk using RAID (Redundant Array of Independent Disks)
 - originally redundant array of inexpensive disks
 - trick : divide and replicate data among multiple drives
- Provides availability, performance and/or capacity.

RAID LEVEL

- Raid 0: block level striping without parity. Zero redundancy. Higher performance and capacity.
- Raid 1: Mirroring without Parity. Higher read performance. Two or more mirrors.
- Raid 2: Bit-level striping with dedicated hamming code parity. Each sequential bit is on a different drive. Not used in practice
- Raid 3: Byte-level striping with dedicated Hamming code parity. Not commonly used.

Raid Level pt2

Raid 4: Block level striping with dedicated parity. Allows I/O requests to be performed in parallel.

Raid 5: block level striping with distributed parity. Masks failure of 1 drive.

Raid 6: block level striping with double distributed parity. Masks up to two failed drives. Better for large drives that take long time to recover.

Parity check:

Use XOR logic

- Key property: $A \oplus B \oplus B = A$
- if we know $A \oplus B$ and B , we can recover A (in case A is lost)

Reliability

- RAID 4 only tolerate one disk failure
 - Data update = overwrite a block + update the parity
 - The parity Disk can be a bottleneck
- in RAID 4, any data update results in parity disks overwrite
- Prevent any parallel disk updates
- increase hard drive failure
 - Parity check is a recovery technique not an error detector
- hard drive itself perform error detection

RAID of RAID

Raid 01: Stripe and then mirror = RAID 0+1

- data is striped across primary disks that are mirrored to secondary disks.

Raid 10: Mirror then stripe = RAID 1+0

The order of digits is the order in which the set is built.

- RAID 0+1 = stripping first and then mirroring
- What is RAID 51?

Synchronous vs Asynchronous

- Synchronous: immediate secondary writes. Write completes only after finishing on secondary storage
 - guaranteed recovery but slow
- Asynchronous: Delayed secondary writes. Writes acked to server even before completion on secondary.
 - writes to secondary are queued in the primary.

Different types of storage:

DAS: Direct attached storage (internal)

- used in majority of PCs and mobile devices: access through OS storage I/O

NAS: Network Attached Storage

- Storage through ethernet; access through OS NIC

iSCSI:

- Network version of SCSI
- Encapsulates native SCSI commands in TCP/IP packets

SAN:

- Storage Area Network: A network whose primary purpose is the transfer of data between storage systems and computer systems

ESCON and FICON:

- Enterprise System connection (ESCON):
 - designed by IBM for mainframes:
 - includes Switches enabling by sharing by multiple users
 - fibers allowed 17 Mbps over 3-43 KM
 - Half-duplex, only one-way at a time (compared to simplex and full-duplex)
- Fiber connectivity (FICON):
 - Supports point-to-point and cascaded topologies (network of switches)
 - supports multiple concurrent I/O operations per channel

- uses single byte command code sets (SBCCS)
- Uses fibre channel as a transport

Fibre Channel:

- ANSI T11 standard for high speed storage area network (SAN)
 - people don't call it network, they call it fabric
- 2,4,6,7,16,32 GBps. Can run on copper or fiber optics
- allows point to point arbitrated loop (ring), switched fabric topologies
- FC host bus adapters (HBA) have a unique 64-bit worldwide name (WWN)
 - similar to 48-bit ethernet MAC addresses OUI, and vendor specific identifiers (VSID)
 - eg. 200000000000C8328FE6
- Why the name Fibre?
 - originally developed to only support fiber optic cabling
 - when copper cabling support was added, ISO decided not to rename the technology.
 - ISO changed to the French spelling to reduce association with fiber optics only medium

Fibre Channel Devices:

- Host Bus Adapters (HBA): Network interface card.
- Gigabit interface converted (GBIC):
 - single mode fiber for long-distance
 - multimode fiber for short distance.
 - HBA ports are empty. Plug in GBIC.
- Switches: A link layer device: Forwards FC frames according to destination address.
- routers and gateways: connect FC to other types of storage (SCSI)

Fibre Channel Layers:

- No universal standards for layers and protocol design
- upper layer protocols
- FC-4: protocol mapping
- FC-3: RAID, encryption
- FC-2: Network Layers
 - Fibre channel shortest path (FSPF) protocol is used to find routes through the fabric.
 - it is a link-state protocol
- FC-1: encoding
- FC-0: Cables, connectors

Fibre Channel Classes of Service:

Class 1: connection oriented dedicated (physical links). Frame order guaranteed. Delivery confirmation. end to end flow control.

Class 2: Connectionless with acknowledgement. Multiple paths

- order not guaranteed. hop by hop and end to end flow control

Class 3: Datagram service. No delivery confirmation.

- only hop by hop flow control. most common.

Class 4: Connection-oriented virtual circuits (fractional links) with delivery confirmation.

- bandwidth control

Class 5: Not yet identified

Class 6: connection-oriented multicast with delivery confirmation

- one to many communication

Class F: packet-switched delivery with confirmation. Inter switch communication only.

Fibre Channel vs iSCSI

Fibre Channel

- the current market leader for shared storage technologies
- Provides the highest performance levels; designed for mission-critical applications
- cost of components is relatively high
- relatively difficult to implement and manage

iSCSI

- relatively new, but usage is increasing rapidly
- performance can approach fibre channel speeds; a better fit for databases than NAS; a good fit for small to medium size businesses
- relatively inexpensive compared to fibre channel
- relatively easy to implement and manage

How to separate one huge logical disks into multiple logical disk?

Storage virtualization:

- Storage virtualization means that applications can use storage without any concern for where it resides, what the technical interface is, how it has been implemented, which platform it uses, and how much of it is available.
- Distance: Remote storage devices appear local
- Size: Multiple smaller volumes appear as a single large volume
- spread: data is spread over multiple physical disks to improve reliability and performance
- file system: Windows, Linux, and UNIX all use the same storage devices

- Virtual interface: A SCSI disk connected to a computer with no SCSI Interface

Virtual Storage Area Network (VSAN)

- Zones in a FC SAN provide isolation among tenants. Some switch ports can see only some other switch ports.
- Different zones share a zone server, name server, and login server
 - subject to common failures
- Virtual Storage Area Network (VSAN) technology allows different partitions with their own servers. Similar to VLANs.
- Each VSAN provide complete fabric services
 - each VSAN can be subdivided in to zones.

Virtual Storage Area Network:

- In-Band: All control and data goes to virtualization appliance
- Out-band: All control goes to namespace root. Data goes directly between host and storage

In band virtualization:

Advantages:

- simple, centralized control - one device sees all I/O
- Easy to implement features: caching, encryption, dedup, replication, - all in one place
- Transparent to hosts - no special software or drivers needed
- Straightforward to deploy and manage

Disadvantages:

- single point of failure - if the appliance dgoes down, all I/O stops
- Bottleneck - all data must flow through one device; limits total bandwidth
- requires high-availability clustering of the appliance itself (adds cost)
- appliance must keep up with aggregate I/O of all hosts

Best Used for mid-size deployments:

- manageability and feature richness matter more than raw throughput.

Out of band virtualization:

Advantages

- higher throughput - data never touches the appliance
- no single bottleneck in the data path; scales more easily
- appliance failure does not interrupt active I/O (data path still intact)
- lower latency for large sequential workloads

Disadvantages:

- Metadata consistency is hard - namespace server must stay perfectly in sync with actual storage state
- requires host-side software/agents to query the namespace server
- advanced features (caching, encryption) are harder to add - no central point to hook into
- more complex to implement and troubleshoot.
- best used when performance and scale are the priority

BIIGGG jump - Storage management March 30th

Virtual Storage

- Partitions and file systems are "virtual" views of the storage
 - a disk array can be partitioned into virtual (logical) devices with logical unit numbers, file systems assigned to different tenants
- Thin provisioning: Allocate blocks only when used.
- Uses multiple physical disks to look like a single disk using RAID (Redundant Array of independent disks)
 - originally redundant array of inexpensive disks (as invented by patterson, Gibson, and katz)
 - trick: divide and replicate data among multiple devices
- provides availability, performance and/or capacity.

Synchronous vs Asynchronous

- Synchronous: immediate secondary writes. Write completes only after finishing on secondary storage
 - guaranteed recovery but slow
- Asynchronous: Delayed secondary writes. Writes acked to server even before completion on secondary.
 - writes to secondary are queued in the primary.

Fibre Channel:

- ANSI T11 standard for high speed storage area network (SAN)
 - people don't call it network, they call it fabric
- 2,4,6,7,16,32 GBps. Can run on copper or fiber optics
- allows point to point arbitrated loop (ring), switched fabric topologies
- FC host bus adapters (HBA) have a unique 64-bit worldwide name (WWN)

- similar to 48-bit ethernet MAC addresses OUI, and vendor specific identifiers (VSID)
- eg. 200000000000C8328FE6
- Why the name Fibre?
 - originally developed to only support fiber optic cabling
 - when copper cabling support was added, ISO decided not to rename the technology.
 - ISO changed to the French spelling to reduce association with fiber optics only medium

Fibre Channel Layers:

- No universal standards for layers and protocol design
- upper layer protocols
- FC-4: protocol mapping
- FC-3: RAID, encryption
- FC-2: Network Layers
 - Fibre channel shortest path (FSPF) protocol is used to find routes through the fabric.
 - it is a link-state protocol
- FC-1: encoding
- FC-0: Cables, connectors

Fibre Channel Classes of Service:

Class 1: connection oriented dedicated (physical links). Frame order guaranteed. Delivery confirmation. end to end flow control.

Class 2: Connectionless with acknowledgement. Multiple paths

- order not guaranteed. hop by hop and end to end flow control

Class 3: Datagram service. No delivery confirmation.

- only hop by hop flow control. most common.

Class 4: Connection-oriented virtual circuits (fractional links) with delivery confirmation.

- bandwidth control

Class 5: Not yet identified

Class 6: connection-oriented multicast with delivery confirmation

- one to many communication

Class F: packet-switched delivery with confirmation. Inter switch communication only.

Fibre Channel vs iSCSI

Fibre Channel

- the current market leader for shared storage technologies
- Provides the highest performance levels; designed for mission-critical applications

- cost of components is relatively high
- relatively difficult to implement and manage iSCSI
- relatively new, but usage is increasing rapidly
- performance can approach fibre channel speeds; a better fit for databases than NAS; a good fit for small to medium size businesses
- relatively inexpensive compared to fibre channel
- relatively easy to implement and manage

Virtual Storage Area Network (VSAN)

- Zones in a FC SAN provide isolation among tenants. Some switch ports can see only some other switch ports.
- different zones share a zone server, name server, and login server.
 - subject to common failures
- Virtual Storage Area Network (VSAN) technology allows different partitions with their own servers. Similar to VLANs.
- Each VSAN provide complete fabric services
 - each VSAN can be subdivided into zone

Virtual Storage Area Network:

In-Band: All control and data goes to virtualization appliance.

Out-Band: All control goes to Namespace root. Data goes directly between host and storage

In band virtualization:

Advantages:

- simple, centralized control - one device sees all I/O
- Easy to implement features: caching, encryption, dedup, replication, - all in one place
- Transparent to hosts - no special software or drivers needed
- Straightforward to deploy and manage

Disadvantages:

- single point of failure - if the appliance goes down, all I/O stops
- Bottleneck - all data must flow through one device; limits total bandwidth
- requires high-availability clustering of the appliance itself (adds cost)
- appliance must keep up with aggregate I/O of all hosts

Best Used for mid-size deployments:

- manageability and feature richness matter more than raw throughput.

Out of band virtualization:

- Advantages
 - higher throughput - data never touches the appliance
 - no single bottleneck in the data path; scales more easily
 - appliance failure does not interrupt active I/O (data path still intact)
 - lower latency for large sequential workloads
- Disadvantages:
 - Metadata consistency is hard - namespace server must stay perfectly in sync with actual storage state
 - requires host-side software/agents to query the namespace server
 - advanced features (caching, encryption) are harder to add - no central point to hook into
 - more complex to implement and troubleshoot.
- best used when performance and scale are the priority

Virtual File Systems:

- Storage access is either block based or file based.
- file systems: NTFS or FAT32 store files on a block based storage.
 - windows DFS (distributed file system)
 - linux DFS: open source implementation of windows DFS on Linux
 - AFS: Andrew file system from CMU (Andrew Carnegie)
- Virtual file systems allow files located on multiple network drives to appear as if on a single local drive
 - network drives can be replicated, relocated, reconstructed
- Parallel Virtual File System (PVFS) distributes data across multiple servers to provide concurrent access for parallel tasks

Wed April 1

Exam 3: April 6

Exam 4: April 29 (cloud security)

Optional lecture May 6th (last day of class)

Poster pres = May 4th

Final Exam May 13 3:15 CK150 (in here) makeup file = send email with final schedule if there's a conflict (May 13)

HW 8 due today HW 9 release tmr

Distributed File System:

- A distributed file system, or DFS, is a data storage and management scheme that allows users or applications to access data files from shared storage.
 - Location transparency - users see a single namespace for all the data files, regardless of which computer they're using to access or store the files. Users won't be able to tell where the file was initially stored and can move files around within the folders as needed without having to change the path name.
 - Redundancy - through a file replication feature, DFS spreads copies of a file across the nodes of the cluster, which means data stays highly available, even in the event of a server failure.
- Key features:
 - Access transparency - users access files as if they are stored locally on their own workstations
 - Location transparency - host machines don't need to know where the file data actually is because the DFS manages that.
 - File locking - the system locks down files in use across locations to prevent two users from different locations making changes to the same file at the same time.
 - Encryption for data in transit - DFS protects data by encrypting it as it moves through the system

Design principles:

- Component failures are the norm rather than the exception
 - don't use high-end machines, instead tolerate failures when they happen
- Files are huge (multi-GB files are common)
 - high sustained bandwidth is more important than low latency
 - place more priority on processing data in bulk
- Most files are mutated by appending new data rather than overwriting existing data
- files are typically written once, then spread sequentially
- Co-designing application and file system API
 - GFS has a relaxed consistency model for simpler design
 - atomic append to allow multiple clients to append to the same file

GFS Interfaces:

- Standard file operations
 - create, delete, open, close
 - read, write
- GFS-specific operations
 - snapshot - create a copy of a file or directory tree at low cost using copy on write
 - record append - appends record to the file automatically (execute without interruption)

GFS Architecture:

- Single coordinator and set of chunkservers
- files are divided into fixed-size large (64MB) chunks
 - each chunk has an immutable and globally unique 64 bit chunk handle assigned by the coordinator
 - each chunk has a version number
 - each chunk is stored at 3 different chunk servers by default
- Coordinator maintains all file system metadata
 - namespace, access control information, mapping from files to chunks, locations of chunks

The Master Node:

- Responsible for all system-wide activities
 - managing chunk leases, reclaiming storage space, load-balancing
- Maintains all file system metadata.
 - Namespaces, ACLs, mapping from files to chunks, and current locations of chunks
 - all kept in memory, namespaces and file-to-chunk mappings are also stored persistently in operation log
- Periodically communicates with each chunkserver in HeartBeat messages
 - this lets master determine chunk locations and assesses state of overall system
 - the chunkserver has the final word over what chunks it does or does not have on its own disks - not the master

Why Single Master?

- The master now has global knowledge of the whole system, which drastically simplifies the design
- but the master is (hopefully) never the bottleneck
 - Clients never read and write file data through the master, client only requests from master which chunkservers to talk to
 - master can also provide additional information about subsequent chunks to further reduce latency
 - further reads of the same chunk don't involve the master, either
- Master state is also replicated for reliability on multiple machines, using the operation log and checkpoints
 - If master fails, GFS can start a new master process at any of these replicas
 - "Shadow" masters also provide read-only access to the file system, even when primary master is down

The Master Node

- For the namespace metadata, master does not use any per-director data structures
 - every file and directory is represented as a node in a lookup table, mapping pathnames to metadata. Stored efficiently using prefix compression (<64 bytes per namespace entry)
- Each node in the namespace tree has a corresponding read-write lock to manage concurrency.
 - Because all metadata is stored in memory, the master can efficiently scan the entire state of the system periodically in the background
 - master's memory capacity does not limit the size of the system

Why Single Master?

- The master now has global knowledge of the whole system, which drastically simplifies the design
- but the master is (hopefully) never the bottleneck
 - Clients never read and write file data through the master, client only requests from master which chunkservers to talk to
 - master can also provide additional information about subsequent chunks to further reduce latency
 - further reads of the same chunk don't involve the master, either
- Master state is also replicated for reliability on multiple machines, using the operation log and checkpoints
 - If master fails, GFS can start a new master process at any of these replicas
 - "Shadow" masters also provide read-only access to the file system, even when primary master is down

Chunks and Chunkservers:

- files are divided into fixed-size chunks, which has an immutable, globally unique 64-bit chunk handle
 - by default, each chunk is replicated three times across multiple chunkservers (users can modify amount of replication)
- Chunkservers store the chunks on local disks as linux files
 - metadata per chunk is <64 bytes (stored in master)
 - current replica locations
 - reference count (useful for copy-on-write)
 - version number (for detecting stat replicas)

Chunk Size:

- 64 MB, a key design parameter (much larger than most file systems)
- Disadvantages:
 - Wasted space due to internal fragmentation
 - small files consist of a few chunks, which then gets lots of traffic from concurrent clients
 - this can be mitigated by increasing the replication factor
- Advantages:
 - Reduces clients' need to interact with master (reads/writes on the same chunk only require one request)
 - since client is likely to perform many operations on a given chunk, keeping a persistent TCP connection to the chunk server reduces network overhead
 - Reduces the size of the metadata stored in master -> metadata can be entirely kept in memory

Read Workflow

- Client sends <Name, offset> to Master
- Master sends <handle, a list of chunkservers, version#> to the client, after the client caches this information
- Client picks one chunkserver and sends <handles, offset>
 - pick closest server to minimize network overheads
 - check version#
- Chunkserver reads data from disk to sends content to client

Write Workflow

- use primary back replication to create a 3-copy replicated system
- This means clients need to know which chunkserver is the primary
 - primary will order write requests from multiple clients
 - Coordinator needs to pick a chunkserver as the primary, how? Pick one that is most up-to-date? How to tell which server is up-to-date?
 - Is it possible that two chunkservers believe that they are the primary?
- When can primary reply to the client? Use lease!

Write Workflow

1. client asks master for all chunk servers (including all secondaries)
2. Master grants a new lease on chunk, increase the chunk version number, tells all replicas to do the same. Replies to client. Client no longer has to talk to master
3. Client pushes data to all servers, not necessarily to primary first

4. Once data is acked, client sends write request to primary. Primary decides serialization order for all incoming modifications and applies them to the chunk.
5. After finishing the modification, primary forwards write request and serialization order to secondaries, so they can apply modifications in the same order. (If primary fails, this step is never reached.)
6. All secondaries reply back to the primary once they finish the modifications
7. Primary replies back to the client, either with success or error.
 - If write succeeds at primary but fails at any of the secondaries, then we have inconsistent state -> error returned to client
 - Client can retry steps (3) through (7)

Append Workflow

- Same as before, but the following extra steps:
 - in step 4 primary checks to see if appending record to current chunk would exceed max size
 - If so, pads the chunk, notifies secondaries to do the same, and tells client to retry request on next chunk
- If record append fails at any of the replicas, the client must retry
 - This means that the replicas of the same chunk may contain duplicates
- A successful record append = the data must have been written at the same offset on all replicas of the chunk
 - Hence, GFS guarantees that record append will be defined interspersed with inconsistent

~15 years later

- Scale is much bigger
 - now 10k servers instead of 1k, 100 PB instead of 100TB
- Bigger change: updates to small files
- Around 2010: incremental update of the google search index
- GFS -> Colossus

Apache Hadoop - open source version of google file system

Questions on the Exam will be similar to;

Container reduces the system overhead by removing the guest operating system
t/f

Which of the following is packaged in a container?

Group of answer choices

Application code

User data

Libraries/Dependencies

System utilities

All of the above

List three advantages of Kubernetes over Docker Swarm.

free write

In your own words, explain the difference between Docker and a container.

free write

SCSI and ATA are protocols designed for which of the following applications?

Group of answer choices

DAS

NAS

SAN

In your own words, describe what RAID 15 is.

Asynchronous replication delays secondary writes to backup disks, which improves write performance but risks data integrity if disks fail before the backups are updated.

t/f

In your own words, explain the difference between “fiber” and “fibre.”

free write

Fibre Channel exclusively supports fiber optic transmission.

True

False

How many bits does a WWN (World Wide Name) consist of?

quick answer